

MULUNGUSHI UNIVERSITY ZAMBIA

SCHOOL OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

ICT261 – CUSTOMER MANAGER CLASSROOM LAB

Student Name	ANDREW MOYO
Student Number	202203671
Programme	Bachelor of Computer Science and Software Engineering
Course	ICT261
Topic	Customer Manager using JavaFX
Date	6 October 2026

1. Introduction

This practical involves developing a Customer Manager application using JavaFX. The application allows a user to enter a customer's name, select a province, add the customer to a table, validate incorrect input, delete a selected customer after confirmation, and use the keyboard to operate the application.

2. Lab Requirements

1. Build a form with a name field and province list.
2. Create a Customer class and `ObservableList<Customer>`.
3. Add a `TableView` with name and province columns.
4. Validate input, then add the customer.
5. Confirm deletion of a selected customer.
6. Test invalid input and keyboard access.

3. Complete JavaFX Code

```
import javafx.application.Application;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
import java.util.Optional;

public class CustomerManager extends Application {

    private final ObservableList<Customer> customers =
        FXCollections.observableArrayList();

    @Override
    public void start(Stage stage) {

        Label nameLabel = new Label("Customer Name:");
        TextField nameField = new TextField();
        nameField.setPromptText("Enter customer name");

        Label provinceLabel = new Label("Province:");
        ComboBox<String> provinceBox = new ComboBox<>();

        provinceBox.getItems().addAll(
```

```

        "Central", "Copperbelt", "Eastern", "Luapula",
        "Lusaka", "Muchinga", "Northern",
        "North-Western", "Southern", "Western"
    );
    provinceBox.setPromptText("Select province");

    Button addButton = new Button("Add Customer");
    Button deleteButton = new Button("Delete Customer");
    Button clearButton = new Button("Clear");

    TableView<Customer> table = new TableView<>();

    TableColumn<Customer, String> nameColumn =
        new TableColumn<>("Name");
    nameColumn.setCellValueFactory(
        new PropertyValueFactory<>("name"));

    TableColumn<Customer, String> provinceColumn =
        new TableColumn<>("Province");
    provinceColumn.setCellValueFactory(
        new PropertyValueFactory<>("province"));

    table.getColumns().addAll(nameColumn, provinceColumn);
    table.setItems(customers);
    nameColumn.setPrefWidth(250);
    provinceColumn.setPrefWidth(200);

    addButton.setOnAction(event -> {
        String name = nameField.getText().trim();
        String province = provinceBox.getValue();

        if (name.isEmpty()) {
            showAlert(Alert.AlertType.ERROR, "Invalid Input",
                "Please enter the customer name.");
            nameField.requestFocus();
            return;
        }

        if (province == null) {
            showAlert(Alert.AlertType.ERROR, "Invalid Input",
                "Please select a province.");
            provinceBox.requestFocus();
            return;
        }

        customers.add(new Customer(name, province));
        nameField.clear();
        provinceBox.setValue(null);
        nameField.requestFocus();
    });

    deleteButton.setOnAction(event -> {
        Customer selectedCustomer =
            table.getSelectionModel().getSelectedItem();

        if (selectedCustomer == null) {
            showAlert(Alert.AlertType.WARNING,
                "No Customer Selected",
                "Please select a customer to delete.");
            return;
        }

        Alert confirmation =
            new Alert(Alert.AlertType.CONFIRMATION);
        confirmation.setTitle("Confirm Deletion");
        confirmation.setHeaderText("Delete Customer");
        confirmation.setContentText(
            "Are you sure you want to delete "
            + selectedCustomer.getName() + "?");

        Optional<ButtonType> result =
            confirmation.showAndWait();

        if (result.isPresent()
            && result.get() == ButtonType.OK) {
            customers.remove(selectedCustomer);
        }
    });

    clearButton.setOnAction(event -> {
        nameField.clear();
        provinceBox.setValue(null);
        nameField.requestFocus();
    });

    nameField.setOnKeyPressed(event -> {
        switch (event.getCode()) {
            case ENTER:
                addButton.fire();
                break;
            case TAB:
                provinceBox.requestFocus();

```

```

        break;
    default:
        break;
    }
});

provinceBox.setOnKeyPressed(event -> {
    if (event.getCode() == javafx.scene.input.KeyCode.ENTER) {
        addButton.fire();
    }
});

GridPane form = new GridPane();
form.setHgap(10);
form.setVgap(10);
form.setPadding(new Insets(10));

form.add(nameLabel, 0, 0);
form.add(nameField, 1, 0);
form.add(provinceLabel, 0, 1);
form.add(provinceBox, 1, 1);
form.add(addButton, 0, 2);
form.add(deleteButton, 1, 2);
form.add(clearButton, 2, 2);

VBox root = new VBox(10);
root.setPadding(new Insets(15));
root.getChildren().addAll(form, table);

Scene scene = new Scene(root, 650, 450);
stage.setTitle("Customer Manager - Andrew Moyo");
stage.setScene(scene);
stage.show();

nameField.requestFocus();
}

private void showAlert(Alert.AlertType type,
    String title,
    String message) {
    Alert alert = new Alert(type);
    alert.setTitle(title);
    alert.setHeaderText(null);
    alert.setContentText(message);
    alert.showAndWait();
}

public static class Customer {
    private String name;
    private String province;

    public Customer(String name, String province) {
        this.name = name;
        this.province = province;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getProvince() {
        return province;
    }

    public void setProvince(String province) {
        this.province = province;
    }
}

public static void main(String[] args) {
    launch(args);
}
}

```

4. Explanation of the Solution

Requirement 1: Form with Name and Province

A TextField is used to enter the customer's name and a ComboBox is used to select the province. The province list contains all ten provinces of Zambia.

Requirement 2: Customer and ObservableList

The Customer class stores the customer's name and province. An ObservableList stores all customers and is connected to the TableView.

Requirement 3: TableView

The TableView contains two columns: Name and Province. The columns obtain their values from the Customer object's getName() and getProvince() methods.

Requirement 4: Validate and Add Customer

The program checks that the name is not empty and that a province has been selected. Only valid data is added to the ObservableList.

Requirement 5: Confirm Deletion

The user selects a customer from the table. A confirmation dialog asks whether the selected customer should be deleted. The customer is removed only after OK is selected.

Requirement 6: Invalid Input and Keyboard Access

The application displays error messages for invalid input. ENTER can activate Add Customer, TAB moves focus to the province field, and requestFocus() manages keyboard focus.

5. Testing

Test	Action	Expected Result
1	Leave name empty and click Add Customer	Error: Please enter the customer name.
2	Enter name but do not select province	Error: Please select a province.
3	Enter Andrew Moyo and select Central	Customer is added to the table.
4	Select a customer and click Delete	Confirmation dialog appears.
5	Click OK in confirmation dialog	Selected customer is deleted.
6	Press ENTER in name/province field	Add Customer is activated.
7	Press TAB from name field	Focus moves to province field.

6. Example Test Data

Customer Name	Province
Andrew Moyo	Central
John Banda	Lusaka
Mary Phiri	Copperbelt
Grace Mwansa	Eastern
Peter Zulu	Southern

7. Requirement Checklist

Requirement	Status
Name field	Completed
Province list	Completed
Customer class	Completed
ObservableList<Customer>	Completed
TableView	Completed
Name and Province columns	Completed
Input validation	Completed
Add customer	Completed
Delete confirmation	Completed
Invalid input testing	Completed
Keyboard access	Completed

8. Conclusion

The Customer Manager application successfully implements all six requirements in the ICT261 classroom lab. It provides a JavaFX interface for entering, validating, displaying and deleting customers, while also supporting keyboard access.

Student: ANDREW MOYO Student Number: 202203671